# Letting AI Formalize the Perceptron Mistake Bound

Johann Birnick

15 April 2026, UCSD

Formal Learning Theory Course by Prof. Tianhao Wang

# Recall: Perceptron Algorithm

Lecture 1: https://teach-learning-theory.tianhaowang.com/notes/notes_1#linear

ALGORITHM 2 **Perceptron (Rosenblatt 1958)**

**Input:** Feature map $\phi : \mathcal{X} \to \mathbb{R}^d$. Labels $y_t \in \{-1, +1\}$.

1. Initialize `w_1 = 0`.
2. Receive $x_t$, predict $\hat{y}_t = +1$ if $\langle w_t, \phi(x_t) \rangle \geq 0$, and predict $\hat{y}_t = -1$ otherwise.
3. Receive true label $y_t$.
4. If $\hat{y}_t \neq y_t$ (mistake): set `w_{t+1} = w_t + y_t * phi(x_t)`.
5. Else: set `w_{t+1} = w_t`.

# Perceptron Mistake Bound

THEOREM 4 **Perceptron Mistake Bound (Novikoff 1962)**

If the data is linearly separable with margin $\gamma > 0$ and $\|\phi(x_t)\| \leq R$ for all $t$, then the Perceptron algorithm makes at most

$$M \leq \frac{R^2}{\gamma^2}$$

mistakes.

# Perceptron Mistake Bound

THEOREM 4 **Perceptron Mistake Bound (Novikoff 1962)**

If the data is linearly separable with margin $\gamma > 0$ and $\|\phi(x_t)\| \leq R$ for all $t$, then the Perceptron algorithm makes at most

$$M \leq \frac{R^2}{\gamma^2}$$

mistakes.

Let's (let AI) formalize this in Lean 4!

# Perceptron Mistake Bound - Lemmas

**LEMMA 1** **Alignment grows with each mistake**

After $M_t$ mistakes, $\langle w^*, w_{t+1} \rangle \geq \gamma \cdot M_t$.

> ▸ *Proof*

**LEMMA 2** **Norm grows slowly**

After $M_t$ mistakes, $\|w_{t+1}\|^2 \leq R^2 \cdot M_t$.

> ▸ *Proof*

# Perceptron Mistake Bound - Proof

*Proof of Theorem 4.*

Combining the two lemmas with the Cauchy–Schwarz inequality and $\|w^*\| = 1$:

$$\gamma\, M_t \;\le\; \langle w^*, w_{t+1}\rangle \;\le\; \|w^*\| \cdot \|w_{t+1}\| \;=\; \|w_{t+1}\| \;\le\; R\sqrt{M_t}.$$

Dividing both sides by $\sqrt{M_t}$ (assuming $M_t > 0$):

$$\gamma\sqrt{M_t} \;\le\; R \qquad \Longrightarrow \qquad M_t \;\le\; \frac{R^2}{\gamma^2}.$$

□

# Phase I

## *Prepare Project+Context for the AI*

# Creating the Project

```
$ lake +4.28.0 new formalization-perceptron-claude math
```

# Creating the Project

```
$ lake +4.28.0 new formalization-perceptron-claude math
```

- `lake` is the project manager for Lean

# Creating the Project

```
$ lake +4.28.0 new formalization-perceptron-claude math
```

- `lake` is the project manager for Lean
- we use Lean version `4.28.0` because that's what `aristotle` supports (this is from February 2026, the newest version is `4.29.1`)

# Creating the Project

```
$ lake +4.28.0 new formalization-perceptron-claude math
```

- `lake` is the project manager for Lean
- we use Lean version `4.28.0` because that's what `aristotle` supports (this is from February 2026, the newest version is `4.29.1`)
- `new` creates a new project

# Creating the Project

```
$ lake +4.28.0 new formalization-perceptron-claude math
```

- `lake` is the project manager for Lean
- we use Lean version `4.28.0` because that's what `aristotle` supports (this is from February 2026, the newest version is `4.29.1`)
- `new` creates a new project
- `formalization-perceptron-claude` is the project name

# Creating the Project

```
$ lake +4.28.0 new formalization-perceptron-claude math
```

- `lake` is the project manager for Lean
- we use Lean version `4.28.0` because that's what `aristotle` supports (this is from February 2026, the newest version is `4.29.1`)
- `new` creates a new project
- `formalization-perceptron-claude` is the project name
- `math` is a template which automatically adds `mathlib4` as dependency

# Creating the Project

```
$ lake +4.28.0 new formalization-perceptron-claude math
```

- `lake` is the project manager for Lean
- we use Lean version `4.28.0` because that's what `aristotle` supports (this is from February 2026, the newest version is `4.29.1`)
- `new` creates a new project
- `formalization-perceptron-claude` is the project name
- `math` is a template which automatically adds `mathlib4` as dependency

*Learning:* This always takes *multiple minutes*, since it's re-downloading `matlib4` every time, which is multiple GB in size.

# Providing Context and the Informal Proof

I just asked Claude to convert Tianhao's nice website into a Markdown file.

# Providing Context and the Informal Proof

I just asked Claude to convert Tianhao's nice website into a Markdown file.

```
### 1.1 The prediction protocol

We fix an *instance space* $\mathcal{X}$, which is the set of all possible
inputs (images, emails, points in $\mathbb{R}^d$, etc.), and a binary *label
space* $\mathcal{Y} = \{0, 1\}$. A *hypothesis* is any function $h :
\mathcal{X} \to \mathcal{Y}$ that assigns a label to each instance.

> **Definition 1** (Online Prediction Protocol).
> The online prediction protocol proceeds in rounds $t = 1, 2, \ldots, T$:
>
> 1. An instance $x_t \in \mathcal{X}$ arrives.
> 2. The learner predicts $\hat{y}_t \in \mathcal{Y}$.
> 3. The true label $y_t \in \mathcal{Y}$ is revealed.
>
> The *history* available to the learner at the start of round $t$ is
$H_{t-1} = ((x_1, y_1), \ldots, (x_{t-1}, y_{t-1}))$.
```

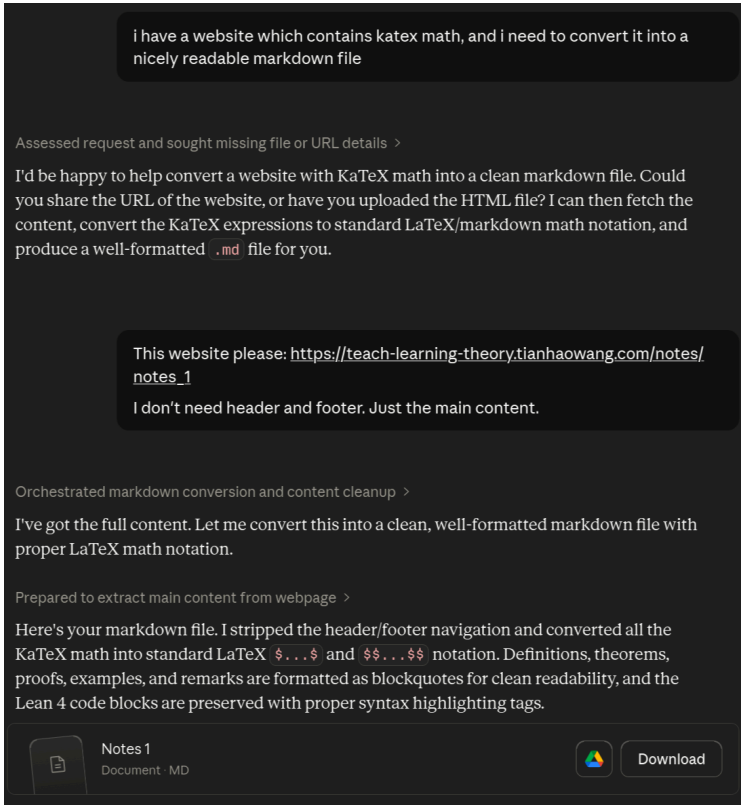
# Providing Context and the Informal Proof

I just asked Claude to convert Tianhao's nice website into a Markdown file.

```
### 1.1 The prediction protocol

We fix an *instance space* $\mathcal{X}$, which is the set of all possible
inputs (images, emails, points in $\mathbb{R}^d$, etc.), and a binary *label
space* $\mathcal{Y} = \{0, 1\}$. A *hypothesis* is any function $h :
\mathcal{X} \to \mathcal{Y}$ that assigns a label to each instance.

> **Definition 1** (Online Prediction Protocol).
> The online prediction protocol proceeds in rounds $t = 1, 2, \ldots, T$:
>
> 1. An instance $x_t \in \mathcal{X}$ arrives.
> 2. The learner predicts $\hat{y}_t \in \mathcal{Y}$.
> 3. The true label $y_t \in \mathcal{Y}$ is revealed.
>
> The *history* available to the learner at the start of round $t$ is
$H_{t-1} = ((x_1, y_1), \ldots, (x_{t-1}, y_{t-1}))$.
```



# Directory Structure

```
formalization-perceptron-claude
├── FormalizationPerceptronClaude
│   └── Basic.lean
├── FormalizationPerceptronClaude.lean
├── lakefile.toml
├── lake-manifest.json
├── lean-toolchain
└── Notes.md       <-- generated from the Lecture 1 website
```

# Making a Prompt

```
I want you to formalize a mathematical proof in Lean4
(version 4.28.0).
```

# Making a Prompt

```
I want you to formalize a mathematical proof in Lean4
(version 4.28.0). The working directory is a new Lean4
project with mathlib4 as a dependency.
```

# Making a Prompt

```
I want you to formalize a mathematical proof in Lean4
(version 4.28.0). The working directory is a new Lean4
project with mathlib4 as a dependency. Read the
Notes.md file. Formalize the Perceptron Mistake Bound
(Theorem 4) which bounds the number of mistakes that
the Perceptron Algorithm (Algorithm 2) can make.
```

# Making a Prompt

```
I want you to formalize a mathematical proof in Lean4
(version 4.28.0). The working directory is a new Lean4
project with mathlib4 as a dependency. Read the
Notes.md file. Formalize the Perceptron Mistake Bound
(Theorem 4) which bounds the number of mistakes that
the Perceptron Algorithm (Algorithm 2) can make. Note
that the proof uses Lemma 1 and Lemma 2. In the
formalized Lean proof, make sure that these two lemmas
are also separate lemmas.
```

# Making a Prompt

I want you to formalize a mathematical proof in Lean4 (version 4.28.0). The working directory is a new Lean4 project with mathlib4 as a dependency. Read the Notes.md file. Formalize the Perceptron Mistake Bound (Theorem 4) which bounds the number of mistakes that the Perceptron Algorithm (Algorithm 2) can make. Note that the proof uses Lemma 1 and Lemma 2. In the formalized Lean proof, make sure that these two lemmas are also separate lemmas. Make sure the code quality closely follows Mathlib standards.

# Phase II
## *Let the AI Work*

# Claude Code (Opus 4.6)

- tried *max* effort, and after a few minutes of thinking…

# Claude Code (Opus 4.6)

- tried *max* effort, and after a few minutes of thinking…

```
You've hit your limit · resets 4pm (America/Los_Angeles)
```

# Claude Code (Opus 4.6)

- tried *max* effort, and after a few minutes of thinking…

```
You've hit your limit · resets 4pm (America/Los_Angeles)
```

- when limit was reset, tried *high* effort, which did the job
- Time: 17min
- Lines of Lean Code: 136 (but many of them are comments)
- ✔ compiles

# Codex CLI (gpt-5.4 xhigh)

- used 90% of the weekly limit (ChatGPT Pro)
- Time: 10min
- Lines of Lean Code: 235
- ✔ compiles

# Gemini CLI (gemini-3.1-pro-preview)

- Time: 10min
- Lines of Lean Code: 93 (+ a bunch of test files)
- ✔ compiles

# Aristotle

```
$ aristotle submit "I want you to [...] follows Mathlib
standards." --project-dir ./ --wait
```

# Aristotle

```
$ aristotle submit "I want you to [...] follows Mathlib
standards." --project-dir ./ --wait
```

```
Project created: 4002c9ff-fae1-4cff-a677-dc13ce1d280f
Project complete! Getting results...
Project saved to 4002c9ff-fae1-4cff-a677-dc13ce1d280f-
aristotle.tar.gz
```

# Aristotle

```
$ aristotle submit "I want you to [...] follows Mathlib
standards." --project-dir ./ --wait
```

```
Project created: 4002c9ff-fae1-4cff-a677-dc13ce1d280f
Project complete! Getting results...
Project saved to 4002c9ff-fae1-4cff-a677-dc13ce1d280f-
aristotle.tar.gz
```

- provided an additional `ARISTOTLE_SUMMARY.md` file
- Time: 36min
- Lines of Lean Code: 143 (but many of them are comments)
- ✔ compiles

# Phase III
## *Evaluate*

# Correct Translation of Statement

- they all generalize to arbitrary inner product spaces

# Correct Translation of Statement

- they all generalize to arbitrary inner product spaces
- Gemini simplifies samples to $z_i = y_i \cdot \phi(x_i)$

# Correct Translation of Statement

- they all generalize to arbitrary inner product spaces
- Gemini simplifies samples to $z_i = y_i \cdot \phi(x_i)$
- Codex is the only one to do a full formalization of the algorithm
  - It doesn't just look at mistake rounds, but formalizes all kind of rounds
  - It even makes a separate datatype for the labels!
  - It's the only one to formalize the feature map $\phi$

# Code Quality

- Aristotle and Claude provided the most comments

# Code Quality

- Aristotle and Claude provided the most comments
- Aristotle, Claude, and Gemini are concise. Codex has more code, but is the only one to implement/formalize the full algorithm. Gemini strips away so much that it might not be obvious that it's equivalent to the natural language statement.

# AI ranking

Claude ranks:

1. Codex
2. Aristotle
3. Claude
4. Gemini

Codex ranks:

1. Codex
2. Claude
3. Aristotle
4. Gemini

Gemini ranks:

1. Codex
2. Aristotle
3. Claude
4. Gemini

# Ideas for Future

- ask for more complicated setups/proofs
  (I want to do the regret bound for Zinkevich's online gradient descent)
- ask for a Lean proof without providing a natural language proof, or even without a natural language statement (just a Lean statement)
- test open source models (normal coding agents)
- test Google's and Math Inc's Lean proving models
  (How to get access?!)
- test open source Lean proving models